

# Designing High-Availability AI Clusters

## Building resilient ML infrastructure that never sleeps

A comprehensive guide for SREs, ML engineers, and infrastructure architects looking to build AI systems that maintain continuous operation in the face of failures.



# What Is High Availability (HA)?

High Availability represents our commitment to **minimize downtime & service disruption** in mission-critical AI systems.

It's traditionally measured as "**nines of uptime**":

99.9%

Three nines

8.76 hours downtime/year

99.99%

Four nines

52.6 minutes downtime/year

99.999%

Five nines

5.26 minutes downtime/year

## Critical for:

-  AI APIs & inference services
-  Recommender systems
-  Fraud detection
-  Healthcare ML
-  Autonomous systems



# HA in AI Context

AI systems present unique high availability challenges that extend beyond traditional applications.



## Model Accessibility

ML models must be **always accessible** for inference requests, regardless of infrastructure state



## Training Resilience

Training pipelines must survive node failures without losing progress or corrupting model artifacts



## Inference Stability

Inference APIs must tolerate crashes & traffic spikes while maintaining SLAs



HA for AI requires comprehensive resilience across **compute + storage + networking** layers

# Core Design Principles

Building truly resilient AI infrastructure requires adherence to four fundamental design principles:

1

## Redundancy

Deploy multiple replicas across zones and regions to eliminate single points of failure

2

## Isolation

Ensure failures in one component don't cascade throughout the system

3

## Failover

Implement automatic rerouting to healthy services when failures occur

4

## Resilience Testing

Regularly conduct chaos drills to validate your HA design assumptions



A well-designed HA system assumes failures will happen and plans for them, rather than treating them as exceptions.

# Compute Layer HA

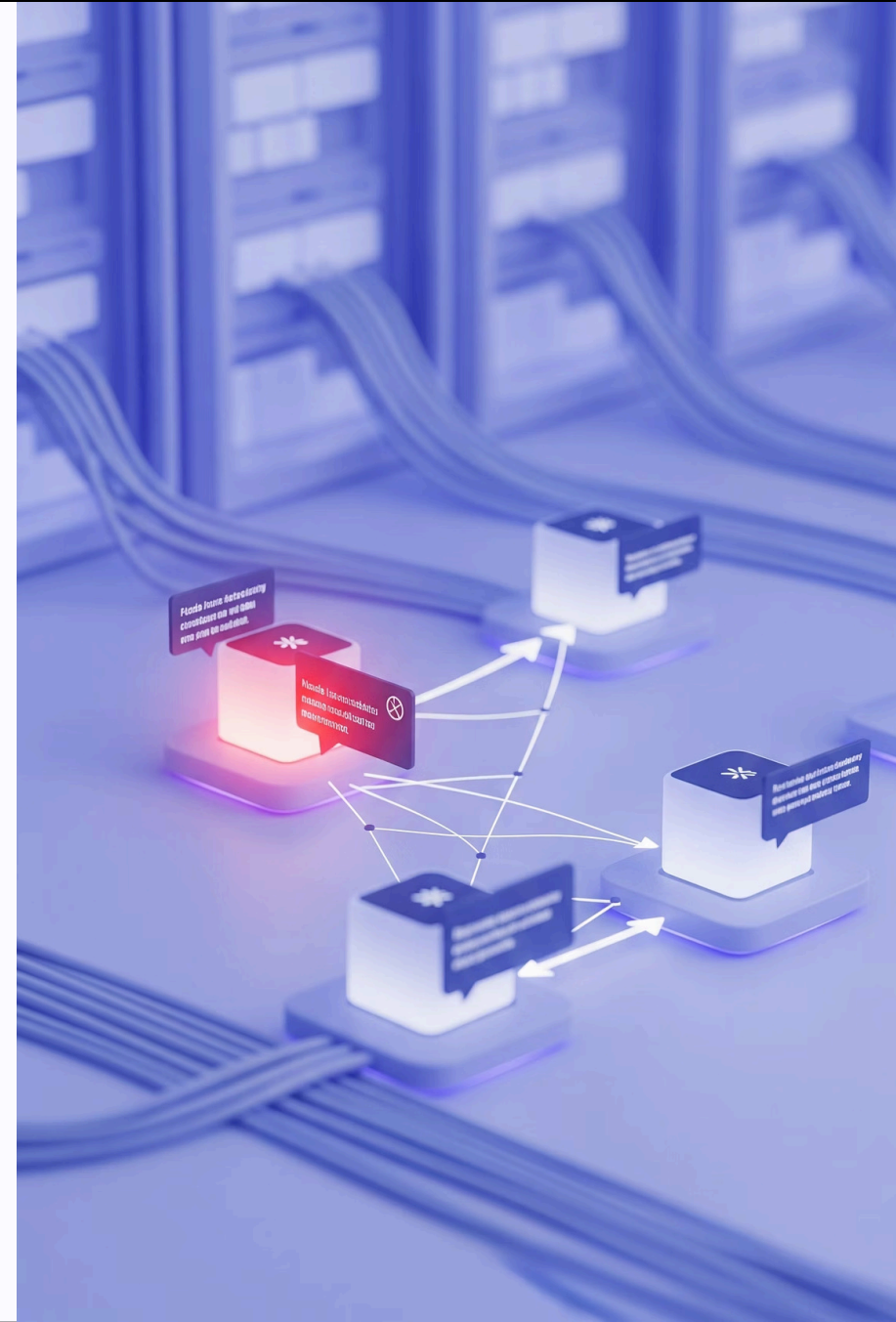
## Kubernetes Foundations

- Deployments with replica sets (3+ minimum)
- **Horizontal/Vertical Pod Autoscalers** (HPA/VPA)
- Pod Disruption Budgets to maintain minimum availability
- Dedicated node pools for critical ML workloads

## Advanced Configurations

- GPU scheduling with node affinity & tolerations
- Pod anti-affinity to spread across zones
- Readiness/liveness probes for auto-healing
- **Multi-zone clusters** → survive data center outage

⚠ **Anti-Pattern:** Overcommitting GPU resources leads to resource contention and reduced availability during failure scenarios.



# Storage Layer HA

## Persistent Storage

- Replicated storage backends (Ceph, EBS, GCP PD)
- Multi-attach volumes where supported
- Storage classes with appropriate redundancy

## ML Data Storage

- Data lakes & vector DBs in **multi-region mode**
- Feature stores with hot replicas
- Distributed file systems (HDFS, GCS, S3)

## Recovery Mechanisms

- Continuous snapshots & backups
- Point-in-time recovery options
- Cross-region replication for critical data

HA storage means not just protecting against hardware failures, but also ensuring consistent performance under load and during recovery operations.



# Network Layer HA



## Intelligent Load Balancing

Deploy load balancers with active **health probes** to detect failures within milliseconds



## Resilient Ingress

Multi-zone ingress controllers (NGINX, Istio) with automatic failover capabilities



## Global Routing

DNS failover across regions (Route53, Cloud DNS) for multi-region resilience



## Dedicated Connectivity

Redundant interconnects for critical ML pipelines with automatic failover

# HA for Training

Distributed training introduces unique availability challenges that require specialized approaches.



## Checkpoint Management

Distributed training with frequent checkpoints for restart on failure without losing progress



## Elastic Training Frameworks

Horovod / DeepSpeed with Kubernetes operators that handle node failures gracefully



## Cost-Effective Resilience

Preemptible/spot instances with fault tolerance mechanisms to balance cost vs. availability

✅ **Best Practice:** Design training jobs that can automatically resume from the last checkpoint when nodes fail, using storage with cross-zone replication.



# HA for Inference

## Multi-Replica Serving

Deploy inference servers (Triton, TorchServe, KServe) with multiple replicas across failure domains

- Minimum 3 replicas per model
- Horizontal scaling based on request patterns

## Safe Deployment Strategies

Implement canary + blue/green rollouts for safe model upgrades with automatic rollback

- Gradual traffic shifting
- Automated quality gates

## Advanced Resilience

Maintain shadow models for resilience testing and implement SLA-driven scaling rules

- Latency-based autoscaling
- Circuit breakers for dependent services

# Observability in HA Clusters

You can't guarantee what you don't measure. Comprehensive observability is essential for validating HA designs.

## Key Metrics to Monitor

- Uptime per service/endpoint
- Failover events and recovery time
- Latency spikes during degraded states
- **Replica count mismatches** or unhealthy nodes
- Resource utilization during failure scenarios

HA validation must be **metrics-driven, not assumption-based**

## Chaos Engineering Integration



Regularly inject failures to validate your HA design:

- Node terminations
- Network partitions
- Disk failures
- Resource exhaustion

# Cost vs Availability

Higher availability requires exponentially greater investment. Match your HA tier to actual business requirements.

## 99.9%

### Three Nines

Single-region, active-passive architecture

- Multi-zone deployment
- Automated failover
- ~8.8 hours downtime/year

## 99.99%

### Four Nines

Multi-region, active-active architecture

- Global load balancing
- Real-time data replication
- ~52 minutes downtime/year

## 99.999%

### Five Nines

Highly engineered redundancy

- Complete infrastructure duplication
- Instant failover mechanisms
- ~5 minutes downtime/year

⊗ Avoid over-engineering for non-critical AI applications. Each additional "nine" typically doubles infrastructure costs.

# Best Practices



## Design for failure as default

Assume components will fail and build redundancy into every layer

## Distribute across failure domains

Spread workloads across [zones/regions](#) to survive localized outages

## Automate recovery processes

Implement automated failover & scaling to minimize human intervention

## Validate with real-world testing

Use [observability](#) + [chaos testing](#) to prove your HA design works

## Right-size your HA investment

Match HA tier to SLA & compliance requirements

# Key Takeaways

## Eliminate Single Points of Failure

Build HA across all three infrastructure pillars:

- Compute: Multi-replica, multi-zone deployments
- Storage: Replicated, consistent data access
- Network: Redundant paths with health checks

## End-to-End Resilience

Both **training & inference** workflows must be resilient:

- Training: Checkpointing, fault tolerance
- Inference: Multi-replica, canary deployments

## Prove It Works

Observability + chaos = evidence, not assumptions:

- Monitor the right metrics
- Test failure scenarios regularly

## Strategic Investment

Balance **cost vs uptime guarantees** based on business impact:

- Critical ML systems: highest availability
- Development systems: cost optimization

Questions?